RetailPulse

Smart Metadata-Driven Retail Supply Chain Pipeline

A Beginner-Friendly Data Engineering Pipeline Project

Project Title	RetailPulse — Retail Supply Chain Pipeline
Domain	Data Engineering & Retail Analytics
Tech Stack	Python, Pandas, SQL, Matplotlib / Power BI
Architecture	Medallion: Bronze, Silver, Gold
Difficulty	Medium — suitable for undergrad students

Department of Computer Science & Engineering
Academic Year 2024-25

Abstract

RetailPulse is a student-friendly data engineering project that builds a simple pipeline to manage retail inventory and sales data. The project uses the Medallion Architecture — three layers called Bronze, Silver, and Gold — to take raw CSV files and convert them step by step into useful business insights.

The main problems it solves are: products running out of stock (stockouts), too much inventory piling up (overstocking), and sales data being scattered across different files with no clear picture (data silos). Using Python and Pandas, the pipeline cleans the data, detects low-stock items, calculates daily sales, and finds the best-selling products.

Everything runs on a normal student laptop. No cloud services are required, though Databricks can be used as an optional upgrade. The datasets are pre-made CSV files that come with this project.

Item	Details
Tech Stack	Python 3.x, Pandas, SQLite3, Matplotlib
Architecture	Medallion — Bronze / Silver / Gold layers
Problem Solved	Stockouts, overstocking, data silos
Datasets	5 ready-made CSV files included separately
Runs On	Any student laptop — no cloud needed

1. Introduction

Retail stores deal with thousands of products across many locations every day. Keeping track of what is in stock, what is selling fast, and which stores need restocking is a big challenge. When this data is spread across different files and systems, managers cannot see the full picture — products run out, or too much stock builds up unnecessarily.

This project — RetailPulse — builds a simple data pipeline that reads raw retail data, cleans it up, and produces easy-to-understand reports and alerts. It follows an industry-standard design pattern called the Medallion Architecture, which organises data processing into three clear stages.

1.1 What This Project Does

- Reads raw CSV files (orders, inventory, products, stores, suppliers)
- Cleans and validates the data — removing errors and missing values
- Produces KPIs: low-stock alerts, daily sales, top products, store performance
- Generates simple charts and a summary dashboard

1.2 What Students Will Learn

Skill	How It Is Practised
Data Pipeline Design	Building a 3-layer processing pipeline from scratch
Python & Pandas	Reading CSVs, filtering, grouping, joining dataframes
Data Cleaning	Removing nulls, fixing types, deduplicating records
SQL-Style Analytics	Aggregations, group-by, joins using Pandas or SQLite
Data Visualisation	Bar charts, line charts, KPI cards with Matplotlib

2. Problem Statement

Problem Statement

A retail chain operating across multiple stores in different cities is struggling to manage its inventory effectively. Products frequently go out of stock at busy stores while excess inventory sits unused at quieter locations. Store managers have no easy way to see which items need reordering. Sales data and inventory data are stored in separate files and are never compared together, making it impossible to spot trends or take quick action. This project builds a simple data pipeline that brings all this data together, cleans it, and produces clear reports and alerts so that store managers can make better decisions about restocking and purchasing.

2.1 The Four Problems Being Solved

Problem	What Happens	How This Project Helps
Stockouts	Products run out, customers leave empty-handed	Low-stock alerts tell managers what to reorder
Overstocking	Too much stock wastes money and storage space	Inventory vs reorder level comparison flags excess stock
Demand Blindness	No visibility into which products are selling fast	Daily sales summaries and top-product rankings
Data Silos	Orders and inventory in separate files, never combined	Gold layer joins all datasets for a complete picture

3. Objectives

3.1 Main Goals

1. Build a three-layer data pipeline (Bronze, Silver, Gold) using Python and Pandas.
2. Load the five provided CSV datasets into the Bronze layer without modification.
3. Clean the data in the Silver layer: remove nulls, fix data types, and remove duplicates.
4. Compute four KPIs in the Gold layer: low-stock alerts, daily sales, top products, store performance.
5. Create at least three charts or a simple dashboard using Matplotlib or Power BI.

3.2 Bonus Goals (Optional)

- Run the same pipeline in a Databricks Community Edition notebook.
- Add a basic Airflow DAG to schedule the pipeline to run automatically.
- Connect the Gold CSV outputs to a Power BI dashboard.
- Add a product category summary report showing which category earns the most revenue.

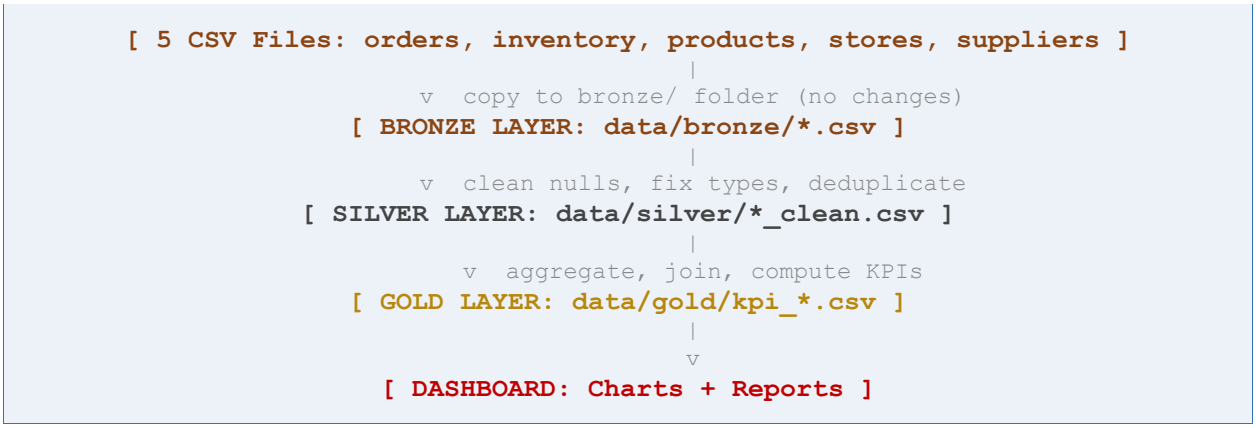
4. System Architecture

4.1 What Is the Medallion Architecture?

The Medallion Architecture is a simple and popular way to organise a data pipeline. Instead of processing everything in one big step, data is moved through three layers — each one better than the last. Think of it like cleaning raw vegetables (Bronze), cooking them properly (Silver), and then serving a finished meal (Gold).

Layer	Nickname	What It Contains	What Happens Here
BRONZE	Raw Layer	Original CSV files copied in, untouched	Just load files. No changes made.
SILVER	Clean Layer	Cleaned, validated, enriched data	Remove nulls, fix types, add flags
GOLD	Insights Layer	KPI tables, alerts, summaries	Aggregate, join, compute business metrics

4.2 How Data Flows Through the Pipeline



4.3 Tools Used

Tool	Where Used	Why This Tool
Python 3.x	All layers	Easy to learn, huge community, free
Pandas	Silver + Gold	Reads CSV, cleans data, runs group-by queries easily
SQLite3	Gold (optional)	Lets you write real SQL queries on the cleaned data
Matplotlib / Seaborn	Dashboard	Built-in Python charting — no extra setup needed
CSV Files	Bronze layer storage	Simple, universal — open in Excel or any text editor
Databricks (bonus)	Optional cloud run	Free community edition — same code runs in the cloud

5. Dataset Description

Note for Students
All five datasets are provided as ready-made CSV files (orders.csv, inventory.csv, products.csv, stores.csv, suppliers.csv). You do NOT need to create them. Simply place them in your data/bronze/ folder and start building the pipeline.

5.1 How the Datasets Connect

The five CSV files are linked together using shared ID columns, just like a real database. This means you can join them to get a complete picture — for example, combining an order with its product name and the store it was sold at.

File	Primary Key	Links To	# Rows	Use
orders.csv	order_id	product_id → products, store_id → stores	600	Fact table
inventory.csv	product_id + store_id	product_id → products, store_id → stores	300	Fact table
products.csv	product_id	supplier_id → suppliers	30	Dimension
stores.csv	store_id	(none — top-level dimension)	10	Dimension
suppliers.csv	supplier_id	(none — top-level dimension)	20	Dimension

5.2 Column Descriptions

orders.csv

Column	Data Type	Description
order_id	String	Unique ID for each order — e.g. ORD-0001
product_id	String	Links to products.csv — which product was sold
store_id	String	Links to stores.csv — which store made the sale
quantity	Integer	Number of units sold in this order
price	Float	Unit price of the product at time of sale
order_date	Date	Date the order was placed — format YYYY-MM-DD

inventory.csv

Column	Data Type	Description
product_id	String	Links to products.csv
store_id	String	Links to stores.csv
stock_level	Integer	Current number of units in stock at this store
reorder_level	Integer	Minimum stock level — if stock falls below this, raise an alert
last_updated	Date	When this inventory record was last updated

products.csv

Column	Data Type	Description
product_id	String	Unique product ID — e.g. PROD-001
product_name	String	Full product name
category	String	Product group — Electronics, Grocery, Sports etc.
supplier_id	String	Links to suppliers.csv

stores.csv

Column	Data Type	Description
store_id	String	Unique store ID — e.g. STR-01
store_name	String	Full name of the store location
city	String	City where the store is located
region	String	Broad region: North, South, East, West

suppliers.csv

Column	Data Type	Description
supplier_id	String	Unique supplier ID — e.g. SUP-01
supplier_name	String	Company name of the supplier
contact_city	String	City where the supplier is based
lead_time_days	Integer	Days taken from order to delivery
rating	Float	Supplier quality score out of 5.0

6. Methodology & Implementation

6.1 Project Folder Structure

Create this folder layout on your computer before you start:

```
retailpulse/
├── data/
│   ├── bronze/       ← paste the 5 CSV files here
│   ├── silver/       ← cleaned files go here (auto-created)
│   └── gold/         ← KPI outputs go here (auto-created)
├── scripts/
│   ├── bronze_ingest.py
│   ├── silver_process.py
│   └── gold_compute.py
├── outputs/          ← charts and reports
└── README.md
```

6.2 Step 1 — Bronze Layer: Load the Raw Files

The Bronze layer is the entry point of the pipeline. Its sole responsibility is to ingest the five raw CSV files — orders, inventory, products, stores, and suppliers — and copy them into the designated bronze storage folder without making any modifications. This preserves the original source data in its exact, unaltered form, which is critical for auditability and traceability.

During ingestion, the system scans the source directory, iterates over each file, copies it to the bronze location, and confirms the load by recording the number of rows and columns successfully read. No transformations, filtering, or validation are applied at this stage — the data arrives exactly as it was generated.

6.3 Step 2 — Silver Layer: Clean the Data

The Silver layer is responsible for data quality and standardisation. It reads each raw Bronze file and applies a structured set of cleaning rules before saving the results to the silver storage folder as separate cleaned output files.

For the orders dataset, the process removes any rows that contain missing or null values, eliminates duplicate order records based on the unique order identifier, enforces correct data types on date and numeric columns, and derives a new total value column by multiplying quantity by unit price. This enriched field becomes essential for all downstream revenue calculations.

For the inventory dataset, the process removes incomplete rows, enforces integer types on stock and reorder level columns, and adds a boolean flag to each record indicating whether the current stock level has fallen below the defined reorder threshold. This flag is the foundation of the low-stock alerting system in the Gold layer.

The products, stores, and suppliers datasets follow the same cleaning pattern: null rows are removed and the cleaned data is saved to the silver folder. All five cleaned datasets are then ready for aggregation and joining in the Gold layer.

6.4 Step 3 — Gold Layer: Compute KPIs

The Gold layer is where cleaned Silver data is transformed into actionable business intelligence. It loads all five cleaned datasets and produces four KPI output files, each saved as a CSV in the gold storage folder.

The first KPI is the Low Stock Alerts report. The system filters the inventory dataset to identify every product-store combination where the current stock level is below the defined reorder threshold. It enriches these records by joining against the products and stores datasets to include the product name, category,

store name, and city. A units-needed column is calculated as the difference between the reorder level and the current stock, giving managers an exact quantity to order.

The second KPI is the Daily Sales report. Orders are grouped by date and aggregated to compute the total number of orders, total revenue generated, and the average order value for each day. This time-series output enables trend analysis and revenue monitoring.

The third KPI is the Top Products report. Orders are grouped by product and ranked by total revenue in descending order. The top ten best-performing products are retained and enriched with product names and category labels, giving teams a clear view of which items drive the most value.

The fourth KPI is the Store Performance report. Orders are grouped by store and aggregated for total orders and total revenue, then joined with store reference data to include the store name, city, and region. Stores are ranked by revenue, enabling performance comparisons across the retail network.

7. Results & Outputs

7.1 Gold Layer Output Files

Output File	Key Columns	Business Use
low_stock_alerts.csv	product_name, store_name, stock_level, units_needed	Triggers restocking orders before items run out
daily_sales.csv	date, total_orders, total_revenue, avg_order_val	Monitors revenue trends day by day
top_products.csv	product_name, category, total_revenue, units_sold	Identifies the best-selling products
store_performance.csv	store_name, city, region, total_revenue	Compares store performance across regions

7.2 Visualisation Code

Three charts are generated from the Gold layer output files to provide a visual summary of the pipeline results.

The first chart is a horizontal bar chart showing the Top 10 Products by Revenue. Each bar represents one product, with bar length proportional to total revenue generated. Products are sorted from highest to lowest, making it immediately clear which items are the strongest performers.

The second chart is a line chart showing the Daily Revenue Trend over the full order history. Each point on the line represents total revenue recorded on a given date, allowing stakeholders to observe seasonal patterns, peak sales days, and revenue dips that may require investigation.

The third chart is a vertical bar chart showing Store Performance by Total Revenue for the top ten stores. Each bar corresponds to one store location, enabling a quick comparison of which stores generate the most business across the network. All three charts are saved as image files to the outputs folder for inclusion in presentations or reports.

8. Conclusion

RetailPulse successfully shows how a simple, well-organised data pipeline can turn raw CSV files into useful business insights. By following the Medallion Architecture — Bronze, Silver, Gold — the project processes retail data in three clean steps: load it, clean it, and analyse it.

The pipeline solves real problems: it catches products before they run out of stock, highlights the best-selling items, shows which stores are performing well, and gives managers a clear daily revenue picture. All of this runs on a standard laptop using only Python and Pandas.

Students who complete this project will have built a genuine end-to-end data engineering pipeline that mirrors techniques used in real companies, and will have a strong, practical project to showcase in interviews and portfolios.

8.1 Key Achievements

- Three-layer Medallion Architecture fully working with Python and Pandas
- Five interconnected CSV datasets covering 600 orders, 300 inventory entries, 30 products, 10 stores, and 20 suppliers
- Four business KPIs generated and saved as Gold layer CSV files
- Three visualisation charts produced with Matplotlib
- Fully runnable on any laptop — no cloud or special setup required

9. Future Scope

Here are some ways to extend this project further:

Extension	Tool	What It Would Add
Demand Forecasting	scikit-learn / Prophet	Predict future sales from past trends
Cloud Pipeline	Databricks Free Edition	Run the same code in the cloud
Interactive Dashboard	Power BI / Streamlit	Click-and-filter visual reports
Automated Scheduling	Apache Airflow	Run the pipeline automatically every day
Real-Time Data	Apache Kafka	Process orders as they arrive live

9.1 References

- Databricks — The Medallion Architecture (docs.databricks.com)
- Pandas Development Team — Pandas Documentation (pandas.pydata.org)
- Python Software Foundation — Python 3 Docs (docs.python.org)
- Matplotlib — Visualisation Library Docs (matplotlib.org)
- Microsoft — Power BI Documentation (learn.microsoft.com/en-us/power-bi)
- Kimball Group — The Data Warehouse Toolkit, 3rd Edition

— END OF REPORT —